

Metode i tehnike testiranja programske podrške

1. Objasnite pojmove i navedite primjere za pojmove pogreška, greška, kvar i incident u programskom inženjerstvu i povežite ih s definiranjem zahtjeva.

Pogreška (error) – kada osoba pogriješi, napravi lošu akciju.

- Primjer: Programer napiše logički pogrešan kod.
- Povezanost s definiranjem zahtjeva: Ako zahtjevi nisu jasno definirani, povećava se mogućnost da će programer nešto krivo napraviti.

Greška (fault) – kada sustav ima grešku ili nedostatak npr. u dizajnu.

- Primjer: Algoritamska pogreška, npr. pogrešno rukovanje s nulom u matematičkim izračunima.
- Povezanost s definiranjem zahtjeva: Neodređenost u zahtjevima u rubnim slučajevima (npr. dijeljenje s nulom) može dovesti do grešaka u kodu.

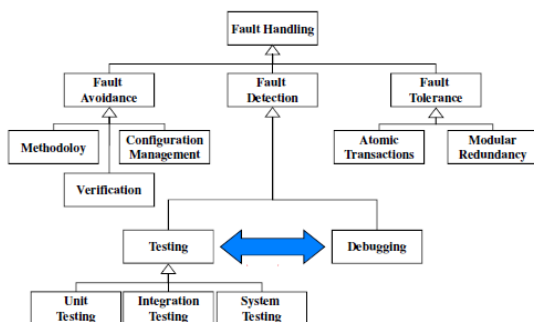
Kvar (failure) – greška u sustavu tijekom rada sustava, odnosi se na pogrešno funkcioniranje ili ponašanje.

- Primjer: Korisnik prilikom unosa podataka u sustav izazove rušenje aplikacije.
- Povezanost s definiranjem zahtjeva: Ako funkcionalni zahtjevi nisu pokrili takve slučajeve, testiranje je otežano, a vjerojatnost za kvar veća.

Incident – vidljivi učinak kvara u sustavu – sustav ima kvar → rezultat kvara je incident.

- Primjer: Korisnik prijavljuje gubitak podataka nakon što se aplikacija sruši.
- Povezanost s definiranjem zahtjeva: Nedostatak zahtjeva za oporavak podataka povećava broj incidenata te zahtjeva daljnju analizu kako bi se problem kasnije riješio.

2. Navedite i ukratko objasnite tehnike rukovanja pogreškama u programskom inženjerstvu.



Rukovanju pogreškama pristupamo na tri načina: izbjegavanje pogrešaka, otkrivanje pogrešaka i tolerancija pogrešaka.

Izbjegavanje pogrešaka za cilj ima spriječiti pojavu grešaka u sustavu tijekom razvoja, a to postiže pomoću korištenja različitih metodologija, upravljanja konfiguracijom (praćenje promjena u kodu, verzija itd.) te verifikacijom (provjera ispravnosti softvera prije testiranja).

Otkrivanje pogrešaka za cilj ima pronaći već postojeće greške u sustavu i to pomoću testiranja (dijelimo na Unit testing, Integration testing i System testing – svi će biti objašnjeni u kasnijim pitanjima) ili debugging-a (proces ispravljanja pogrešaka)

Tolerancija pogrešaka podrazumijeva da se sustavu omogući nastavak rada čak i ako dođe do pogreške, a to se postiže pomoću atomskih transakcija (osiguravaju da se skup operacija izvrši u potpunosti ili poništi kako bi se sustav vratio u prethodno stanje) i

pomoću modularne redundancije koja se odnosi na ugradnju rezervnih modula koji omogućuju ispravan rad sustava u slučaju kvara nekog od modula.

3. Opišite kako se može definirati kvaliteta programske podrške.

Kvaliteta se može definirati na sljedeće načine:

Kompatibilnost sa zahtjevima i potrebama korisnika – jedan je od najbitnijih uvjeta da bi se program odredio kao kvalitetan je da olakšava rad krajnjim korisnicima.

Dobre karakteristike proizvoda kao što su brzina rada, malo zauzeće memorije te brzina pokretanja

Lakoća održavanja i promjena u programu

Kvalitetna dokumentacija, zahtjevi, dizajn

Kompatibilnost sa standardima – podrazumijeva usklađenost s općim standardima (ISO) te usklađenost sa zakonskim regulativama i slično tomu.

Rad u ekstremnim uvjetima s ogromnom količinom podataka te slabim vezama i ograničenim resursima.

4. Objasnite što je testiranje programske podrške, koji su glavni ciljevi testiranja, u kojim se fazama razvoja programa može obavljati i kada završava.

Testiranje programske podrške postupak je vrednovanja cijelog sustava ili neke njegove komponente s ciljem utvrđivanja zadovoljavaju li prethodno definirane zahtjeve.

Ciljevi su testiranja poboljšati kvalitetu sustava, verifikacija i validacija te procjena pouzdanosti samog sustava ili njegovih komponenti.

Testiranje se u raznim oblicima može obaviti **u svakoj fazi razvoja programa** (tijekom faze analize zahtjeva, tijekom dizajniranja kada provjeravamo dizajn, na kraju pisanja izvornog koda kada inače kreće testiranje).

S obzirom da se nijedan sustav ne može smatrati 100% ispravnim, **u teoriji se s testiranjem nikada ni ne završava**, jer za sustav nikada ne garantiramo da je 100% ispravan i testiranje se uvijek može nastaviti. **U praksi, o završetku testiranja s razmatra kada se provjeri što je od zahtjeva isporučeno te jesu li obrađene sve pogreške ili dok ne završe rokovi predviđeni za testiranje.**

5. Ukratko objasnite glavna načela testiranja programske podrške.

Iscrpno ispitivanje nije moguće – jer zahtijeva beskonačne resurse, umjesto toga, potrebna je optimalna aktivnost testiranja na temelju procjene rizika aplikacije.

Testiranje povezanih grešaka (defect clustering) – velika se većina grešaka nalazi u malom broju modula, ali ipak ponavljanje istih testova u istim modulima prestaje otkrivati nove greške nakon određenog vremena.

Povezanost grešaka u modulima (pesticide paradox) – ako se isti testovi ponavljaju, postaju neefikasni jer ne otkrivaju nove pogreške. Dodavanje novih testnih slučajeva osigurava veću učinkovitost testiranja.

Testiranjem dokazujemo postojanost grešaka – testiranje može pokazati da greške postoje, ali ne može dokazati da ih nema, jer je uvijek moguće da je neki nedostatak ostao neotkriven.

Zabluda nepostojanja grešaka – softver bez grešaka može i dalje biti neupotrebljiv ako ne zadovoljava korisničke potrebe. Ispravljanje pogrešaka je beskorisno ako ne ispunjava svoj cilj, stoga testiranje mora obavljati i provjeru ispunjavanja cilja.

Testiranje u ranoj fazi razvoja – Testiranje bi u idealnom slučaju trebalo započeti tijekom definiranja zahtjeva, jer je jeftinije ispraviti pogrešku u ranim fazama.

Testiranje ovisi o kontekstu – Pristup testiranju treba prilagoditi tipu softvera i sustava kojeg kreiramo. Različiti sustavi zahtijevaju različite metode, strategije i vrste testiranja za optimalnu učinkovitost.

6. Objasnite što je verificiranje, a što validiranje, te ih međusobno usporedite.

Verificiranje u kontekstu testiranja za cilj ima provjeriti je li softver ispravno izgrađen prema prethodno definiranim specifikacijama i standardima i fokusira se na internu ispravnost procesa razvoja. Glavne su aktivnosti statička analiza (analiza koda, provjera konzistentnosti) i aktivnosti poput unit testinga, integration testinga i inspekcije koda.

Validiranje za cilj ima provjeriti zadovoljava li softver korisničke potrebe i ciljeve, pa se fokus stavlja na vanjskoj točnosti proizvoda. Koriste se dinamičke analize (user acceptance testing, usability testing) te testiranje u stvarnim uvjetima rada korisnika. Neke se aktivnosti koriste za oba procesa, kao što su system testing i regression testing. Ukratko, verifikacija osigurava da softver tehnički zadovoljava specifikacije, a validacija potvrđuje da softver ispunjava korisničke potrebe.

7. Ukratko objasnite što su osiguranje kvalitete, kontrola kvalitete i testiranje programske podrške.

Osiguranje kvalitete (Quality Assurance) – proces je koji osigurava da svi aspekti razvoja softvera od definiranja zahtjeva do implementacije zadovoljavaju postavljene standarde kvalitete. Fokusira se na prevenciju grešaka i poboljšanje procesa, čime se osigurava da softver bude visoke kvalitete prije nego uđe u fazu testiranja.

Kontrola kvalitete (Quality Control) – odnosi se na aktivnosti koje se provode kako bi se otkrile greške u samom softveru nakon što je razvijen.

Testiranje programske podrške (Software testing) – proces evaluacije softvera kako bi se otkrile pogreške i osigurale njegove funkcionalnosti prema zadanim specifikacijama. U ovu vrstu testiranja spadaju unit testiranje, integracijsko testiranje i funkcijsko testiranje.

8. Navedite i ukratko objasnite aktivnosti procesa testiranja uključujući završetak testiranja i prateće aktivnosti.

Proces testiranja je proces koji obuhvaća velik broj aktivnosti i usko je povezan s procesom razvoja programske podrške. Aktivnosti su:

Planiranje i nadziranje testiranja – aktivnost koja se izvodi tijekom cijelog projekta s ciljem poboljšanja procesa. Definiraju se vrste testova koje će se provesti, metode i strategije testiranja kao i kriterij završetka testiranja. Rezultat planiranja je skup dokumenata koji predstavlja pogled na sustav i aktivnosti.

Analiza i dizajn testova – analizira se sustav i identificiraju se problemi koji će se testirati, a kao rezultat se definira skup testnih slučajeva i procedura.

Implementiranje i izvršavanje testova – proces primjene određenih testnih slučajeva i procedura u skladu s planom i dizajnom.

Prijava i rješavanje problema završetka testiranja – tijekom testiranja izvršavaju se aktivnosti praćenja statusa problema (zapisi testova) i izvješćivanje koje sadrži potvrdu je li program spreman za korištenje.

Završetak testiranja – aktivnost koja uključuje zadaće: provjera što je od zahtjeva isporučeno i jesu li obrađene sve pogreške, završiti i arhivirati testware, pripremiti testware za organiziranje održavanja, vrjednovati testiranja i što se naučilo za buduće projekte.

Prateće aktivnosti – praćenje i kontrola testiranja (skup aktivnosti kojima se prati provedba plana testiranja koja treba osigurati da se rokovi ne prekorače), kontrola kvalitete testiranja (provjera provedbe kontrole kvalitete, je li sustav testiran na dovoljno dobar način), kontrola promjena (prati promjene u sustavu i analiziraju promjene koje bi trebalo uzeti u obzir u testovima), poboljšanje procesa testiranja (skup mjera pomoću kojih se na temelju rezultata kontrole provode određene promjene).

9. Opišite što čini dokumentaciju procesa testiranja programske podrške i koja su glavna obilježja dobrog testnog slučaja.

Dokumentacija procesa testiranja programske podrške obuhvaća niz ključnih elemenata koji osiguravaju da se svi zahtjevi i funkcionalnosti aplikacije ili sustava temeljito testiraju. Glavne komponente ove dokumentacije uključuju plan testiranja, testne slučajeve, matricu izvedivosti i korake poboljšanja testiranja.

Plan testiranja odnosi se na definiranje pristupa testiranju, uključujući vrste testova, tehnike testiranja, metode i slično.

Testni slučajevi predstavljaju specifičan skup koraka koji se poduzimaju za testiranje nekih funkcionalnosti. **Glavna obilježja dobrog testnog slučaja** su jedinstven ID testnog slučaja, proizvodni modul i verzija, namjena i pretpostavka koje jasno definiraju ciljeve testiranja i uvjete koji se moraju ispuniti prije početka testa, detaljan opis koraka koji se poduzimaju u testiranju, očekivani ulaz i izlaz te naknadni uvjeti koji će nastati nakon izvršenja testa. U konačnici, dobar testni slučaj mora biti učinkovit (pronalaži pogreške), primjeren (predstavlja i druge), razvojan (jednostavan za održavati), ekonomičan (jednostavan za korištenje)

Matrica izvedivosti povezuje poslovne zahtjeve, testne scenarije i slučajeve pomažući time u praćenju ispunjavanja zahtjeva tijekom testiranja i identifikaciji nedostataka.

Koraci poboljšanja testiranja uključuju analizu trenutnih metoda, identifikaciju slabih točaka i predlaganje mjera za optimizaciju kako bi se povećala efikasnost i točnost testiranja.

10. Navedite i ukratko opišite elemente testnog plana.

Identifikator testnog plana

Uvod – dijelovi i značajke programa koji će se testirati, planovi projekata te standardi

Dijelovi koji se testiraju – moraju uključivati razinu inačice, način kako se prenose te moraju sadržavati reference na programsku dokumentaciju

Značajke koje se testiraju – utvrditi specifikacije dizajna testa / tehnike testiranja

Značajke koje neće biti testirane – mora se navesti razlog isključivanja pojedine značajke iz testiranja

Pristup – obuhvaća tehnike, aktivnosti i alate. Potrebno je odrediti stupanj pokrivenosti i drugih kriterija završetka, također je potrebno utvrditi ograničenja (npr. rokovi)

Kriterij prolaska/pada komponente – potrebno je opisati koji su kriteriji za prolazak ili pad pojedine komponente

Kriterij završetka i nastavka testiranja – za sve dijelove testnih aktivnosti, potrebno je odrediti koje aktivnosti moraju biti ponovljene kod nastavka testiranja

Isporuke testova – uključuje testni plan, specifikacije dizajna testova, specifikacije testnih slučajeva i procedura, izvješća o prijenosu testnih slučajeva, testni zapis, izvješća o testnim incidentima

Testni zadaci – opisuju zadatak testova te uključuju ovisnost među zadacima i posebne vještine

Okolina – može biti fizikalna, može se odnositi na programe i alate. Opisuje način korištenja, sigurnost u toj okolini i slično

Odgovornosti – jasno se definiraju odgovornosti svih članova tima, uključujući upravljanje, dizajn, pripremu, provedbu, provjeru i rješavanje problema

Potrebe osoblja i educiranje – opis osoblja koje je potrebno za izvršenje testiranja kao i potrebne vještine i znanja

Raspored – opisuje raspored testiranja u kontekstu cijelog projekta

Rizici i nepredvidljivosti – kreiranje plana za nepredvidiva ponašanja za svaki identificirani rizik

Odobrenja – imena i vrijeme odobravanja

11. Ukratko objasnite što je, čemu služi i što obuhvaća matrica izvedivosti, te pokažite neki primjer korištenja.

Matrica izvedivosti je alat koji povezuje poslovne i funkcionalne zahtjeve s pripadajućim testnim slučajevima kako bi se osiguralo da su svi zahtjevi sustava pokriveni testiranjem. Svrha matrice izvedivosti je osigurati da testne aktivnosti odgovaraju zahtjevima, čime se pomaže identificirati nedostajuće testove ili nepotpuno testiranje.

Primjer:

Business Requirement	Test Scenario	Test Case	Defects
BR1: Prijava korisnika	TS1: Provjera prijave s ispravnim podacima	TS1.TC1: Prijava s ispravnim podacima	NIL
	TS2: Provjera prijave s pogrešnim podacima	TS2.TC1: Prijava s pogrešnim korisničkim imenom	D01: Poruka o grešci nije ispravna
		TS2.TC2: Prijava s praznim poljem za lozinku	D02: Polje za unos nije validirano
BR2: Resetiranje lozinke	TS3: Provjera postupka resetiranja lozinke	TS3.TC1: Slanje zahtjeva za resetiranje lozinke	NIL
		TS3.TC2: Resetiranje lozinke s nepostojećim emailom	D03: Poruka o grešci nije prikazana
BR3: Upravljanje korisnicima	TS4: Dodavanje korisnika	TS4.TC1: Dodavanje korisnika s ispravnim podacima	NIL
		TS4.TC2: Dodavanje korisnika bez e-mail adrese	D04: Sustav ne prikazuje grešku

12. Navedite i dajte kratki osvrt na korake poboljšanja testiranja.

Koraci poboljšanja testiranja ključni su za unaprjeđenje procesa testiranja, povećanje kvalitete i efikasnosti testiranja te smanjenje rizika i grešaka u budućnosti. Omogućavaju stalno praćenje i optimizaciju testnih aktivnosti.

Pojednostavljenje procesa nadzora i audita – uključuje unaprjeđenje načina na koji se prate testne aktivnosti, izvještaji i rezultati. Nadzor testiranja mora biti jasan i transparentan kako bi se na vrijeme prepoznali problemi i optimizirali resursi, a time osigurava da su svi testovi ispravno provedeni i dokumentirani.

Uvid u praznine u pokrivenosti – praznine u pokrivenosti se odnose na područja aplikacije koja nisu dovoljno testirana ili nisu uopće testirana. Identifikacija takvih praznina omogućavaju testnom timu da proširi testne slučajeve i poboljša kvalitetu testiranja i time pokrije da su sve funkcionalnosti i zahtjevi pokriveni testovima.

Upravljanje čestim promjenama zahtjeva – promjene u zahtjevima mogu često nastati te je posebno važno te promjene uskladiti s prethodno definiranim planovima i izvedbom testova. Potrebno je kreirati testne planove da budu fleksibilniji kako bi se mogli lakše prilagoditi eventualnim promjenama u zahtjevima. Također, važno je regresijsko testiranje koji osigurava da nove promjene nisu narušile postojeću funkcionalnost, ali je važno i automatizirano testiranje koje se mogu brže ponovno pokrenuti nakon promjenama u zahtjevima, a time se smanjuje vrijeme potrebno za ponovnu validaciju funkcionalnosti.

Praćenje od početka do kraja s alatom: uz neke alate (npr. TM4J) moguće je pratiti testiranje od početka do kraja uključujući sve korake razvoja projekta. Alati mogu olakšati koordinaciju među timovima, stoga je lakše upravljati testnim ciklusima i brže se prepoznaju problemi.

13. Objasnite što je izvješće o greškama u testiranju programske podrške, kako se ono piše i što čini životni ciklus prijavljene greške.

Izvješće o greškama dokument je koji se koristi za praćenje i izvještavanje o problemima koji su se prepoznati tijekom testiranja softvera. Omogućuju timovima da prepoznaju, analiziraju i isprave pogrešku u projektu prije puštanja u produkciju. Izvješće sadrži opis grešaka, korake za reproduciranje greške, očekivani rezultat i aktualni rezultat, serijski broj verzije softverske komponente i prioritet pogreške. **Za pisanje izvješća potrebno je** prvo identificirati pogrešku te ju jasno i precizno opisati uključujući popis koraka koje treba poduzeti kako bi se greška ponovila, opis stvarnog rezultata i onoga koji je očekivan, zatim je potrebno priložiti dokaz koji potvrđuje grešku te je potrebno priložiti informaciju o okruženju u kojem se pojavila pogreška.

Životni ciklus prijavljene pogreške: identifikacija pogreške, verifikacija pogreške (provjera je li riječ o nesporazumu u testiranju ili je zaista greška u sustavu), klasifikacija pogreške, razvoj i ispravak pogreške, testiranje ispravka, zatvaranje pogreška (označava se kao zatvorena u sustavu koji upravlja greškama), ponovno testiranje.

14. Navedite koje glavne faze uključuje životni ciklus razvoja programske podrške, koji glavni modeli razvoja postoje.

Životni ciklus razvoja programske podrške uključuje analizu zahtjeva, dizajn rješenja, programiranje, testiranje i puštanje proizvoda korisnicima

Postoje brojni modeli, ali se **najčešći modeli dijele u grupe:**

Fazni model – aktivnosti razvoja se dijele u faze koje se slijedno izvršavaju (V model, vodopadni model)

Iterativni model – projekt se dijeli na manji broj perioda u kojima se vrše aktivnosti razvoja (Rapid Application Development)

Agilne metode – slične iterativnima, ali na manje formalan način kombiniraju aktivnosti razvoja programa (Agile development)

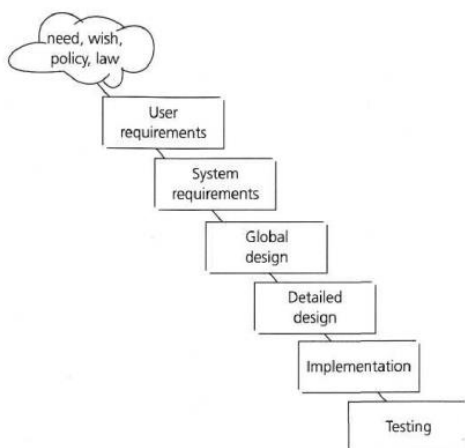
15. Objasnite gdje se u životnom ciklusu razvoja programske podrške nalaze osiguranje kvalitete i testiranje, te što obuhvaća specificiranje zahtjeva.

Osiguranje kvalitete i testiranje ključna je aktivnost u životnom ciklusu razvoja programske podrške, ali se pojavljuje u različitim fazama tog ciklusa. Počinje u analizi zahtjeva, jer je važno od početka definirati standarde kvalitete, strategije i ciljeve testiranja, nastavlja se u dizajnu rješenja gdje se može raditi na pregledu dizajna kako bi se osiguralo da je dizajnirane funkcionalnosti moguće testirati. Tijekom razvoja, tj. programiranja testiranje obuhvaća unit testove gdje se pojedinačne komponente softvera testiraju kako bi se osigurala njihova ispravnost, a u ovoj se fazi mogu početi uključivati i automatizirani testovi. U fazi testiranja se provode različite vrste testiranja i glavna je

dionica životnog ciklusa u pogledu testiranja. U puštanju proizvoda korisnicima, potrebno je osigurati kvalitetu koja se odnosi na provjeru je li proizvod spreman i jesu li svi testovi prošli.

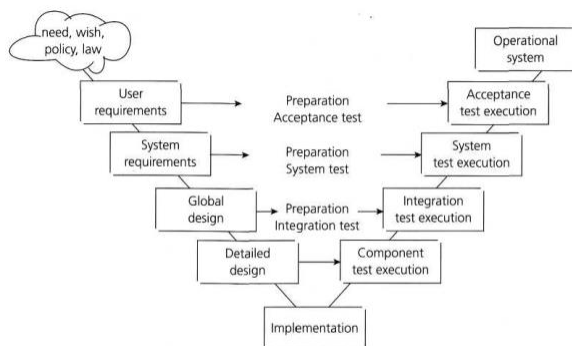
Specificiranje zahtjeva definira što sustav treba raditi i kako će se očekivani rezultati mjeriti, a to obuhvaća poslovne (opisuje kakav problem sustav treba riješiti), funkcionalne (detaljno opisuju specifične funkcionalnosti koje sustav mora pružiti) i nefunkcionalne zahtjeve (aspekti sustava koji nisu direktno vezani uz funkcionalnost, npr. performanse, sigurnost itd.), tehnički zahtjevi (potrebne tehnologije koji omogućuju implementaciju sustava) te korisnički zahtjevi (definiraju što krajnji korisnici očekuju od sustava).

16. Skicirajte i ukratko objasnite testiranje modelom vodopada.



Jedan od prvih modela koji je zasnovan na linearnom pristupu, tj. podjeli na faze koje predstavljaju skupove sličnih aktivnosti. Svaka faza traje dok se ne završe sve aktivnosti u okviru nje i tek tada se prelazi na sljedeću fazu, a faze su zapisane u skici. Testiranje obuhvaća analizu funkcionalnih zahtjeva, definiranje test planova, testiranje programa i prijavi problema. Prednost ovakvog pristupa je što se jasno definiraju faze testiranja programa i to olakšava planiranje testova, ali ako verificiranje utvrdi da sustav ne radi ispravno, projekt se možda može vratiti na početak i značajno kasniti ili propasti. Dakle, jako je osjetljiv model vodopada na povratak u prošle faze.

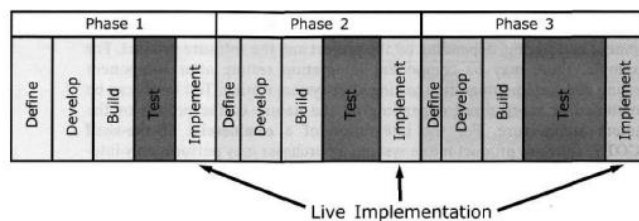
17. Skicirajte i ukratko objasnite testiranje V modelom.



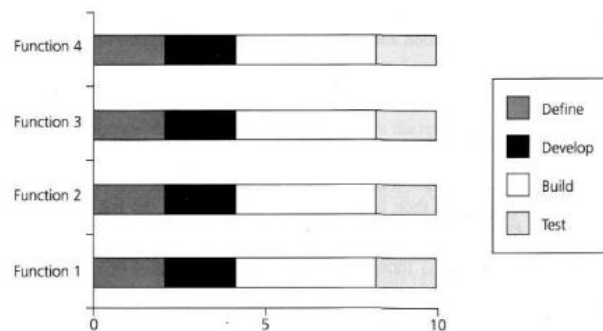
V model je u principu samo modificirani model vodopada u kojem postoji odgovarajuća faza testiranja za svaku fazu razvoja programa, a ispitivanje se vrši paralelno s razvojem. U sljedeću fazu kod ovoga modela moguće je prijeći samo ako je izvršena prethodna faza, ali uz bitna ograničenja, a to su: iz faze analize zahtjeva prelazi se u fazu dizajna samo ako su analizirani zahtjevi i ako su definirani načini kako će se zahtjevi testirati. U

fazu dizajna arhitekture može se prijeći ako je završena faza dizajna sustava i definiran način testiranja cjelokupnog sustava. U fazu dizajna može se prijeći ako je dizajnirana arhitektura i definiran način testiranja komponenti tijekom implementacije. U fazu implementacije može se prijeći ako su dizajnirani moduli koji će biti implementirani te ako je definiran način njihovog testiranja. Iako određenu fazu nije moguće provjeriti, proces provjere može se isplanirati, a time i otkriti probleme u fazi razvoja (npr. ako je definirana lista zahtjeva, počne planiranje testiranja zahtjeva i za neke od zahtjeva se zaključi da su nejasni i ne mogu se testirati, tim se vraća u fazu analize zahtjeva, kako bi se razjasnio zahtjev, za razliku od modela vodopada gdje se sve vraća na početak).

18. Skicirajte i ukratko objasnite iterativni i rapidni ciklus razvoja i testiranja.

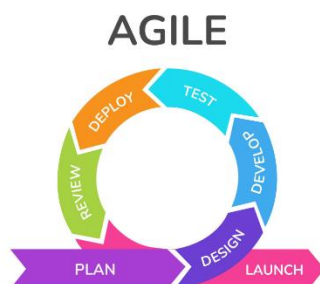


Iterativni ciklus razvoja i testiranja – sastoji se od brojnih malih inkrementalnih faza životnih ciklusa istog projekta koji mogu, umjesto duge vremenske linije razvoja, biti iterativni na razne načine, a služe dodavanju novih funkcionalnosti. Svaki inkrement kao rezultat iteracije može biti testiran kroz jedinične, regresijske, integracijske i druge testove uz opterećenje prilagođeno potrebama testiranja.



Rapid Application Development – formalno je paralelni razvoj komponenata u okviru mini projekata koji su i pojedinačno vidljivi, pa omogućuju reagiranje klijenta, a nakon toga slijedi integracija.

19. Skicirajte i ukratko objasnite testiranje kroz agilni razvoj.



Agilni razvoj softvera iterativni je i inkrementalni pristup u kojem se softver razvija kroz kratke cikluse (sprintove) s ciljem brze isporuke funkcionalnih dijelova sustava. Fokus je na suradnji, prilagodljivosti promjenama i kontinuiranom poboljšanju kroz povratne informacije korisnika.

Testiranje kroz agilni razvoj pristup je testiranju softvera kreiranog agilnim modelom razvoja u kojem se precizno definiraju potrebne faze i aktivnosti. Testne se aktivnosti obavljaju cijelo vrijeme, ne postoji trenutak početka testiranja, jer se analiza zahtjeva i testiranje vrše paralelno s razvojem. Na početku svake faze prije implementacije se proučavaju zahtjevi. Tijekom faze implementacije zahtjeva, paralelno se vrši kreiranje testova. Cijeli se proces prilagođava i po potrebi modificira prema promjenama koje će se dogoditi tijekom projekta.

20. Navedite i ukratko objasnite najznačajnije standarde testiranja programske podrške s naglaskom na ISO/IEC/IEEE 29119.

Najznačajniji standardi testiranja su ISO/IEC/IEEE/29119, ISO/IEC 9126 i ISO/IEC 12119. **ISO/IEC/IEEE/29119** – međunarodni je standard koji definira smjernica za testiranje, a obuhvaća cijeli životni ciklus testiranja, od planiranja, preko dizajna, pa sve to izvještavanja o testnim rezultatima. Standard pruža strukturirani pristup testiranju i uključuje preporuke za različite vrste testiranja. Ključne komponente ovog standarda su testni procesi (specifikacija testiranja, planiranja i izvještavanja), testni dokumenti (smjernica za izradu dokumenata kao što su testni planovi i izvještaji), osiguranje kvalitete (fokusira se na osiguranje kvalitete kroz cijeli proces razvoja softvera, s naglaskom na testiranje)

ISO/IEC 9126 – standard koji definira kvalitativna svojstva softverskih proizvoda. Iako je pomalo zastario, ključan je za postavljanje temelja za razumijevanje kvalitete softvera i njihovih ključnih karakteristika kao što su funkcionalnost, pouzdanost, upotrebljivost itd.

ISO/IEC 12119 – standard koji se odnosi na kvalitetu komercijalnih softverskih paketa, uključujući njihove specifikacije i kriterija prihvatanja. Standard pokriva funkcionalne i sigurnosne zahtjeve, zahtjeve za kompatibilnost, performanse i dokumentaciju.

21. Objasnite razliku između razina testiranja (testing levels), vrsta testiranja (testing types) i metoda testiranja (testing methods).

Razine testiranja fokusiraju se na faze razvoja softvera u kojima se testiranje provodi, a to su testiranje jedinica (unit testing), integracije (integration testing), sustava (system testing) i prihvatanja (acceptance testing), svaka s ciljem provjere različitih aspekata sustava. **Vrste testiranja** se odnose na specifične ciljeve testiranja kao što su funkcionalnosti, performanse, sigurnost i upotrebljivost (performance testing, security testing, usability testing, regression testing, exploratory testing...). **Metode testiranja** definiraju kako se testovi provode, tu spadaju black-box testiranje, white-box testiranje i gray-box testiranje (testiranje ponašanja uz poznavanje koda, bez poznavanja koda ili kombinacijom ta dva pristupa), agile testing, ad hoc testing.

22. Navedite glavne tipove testiranja programske podrške.

Ručno testiranje, automatizirano testiranje, istraživačko testiranje, regresijsko testiranje.

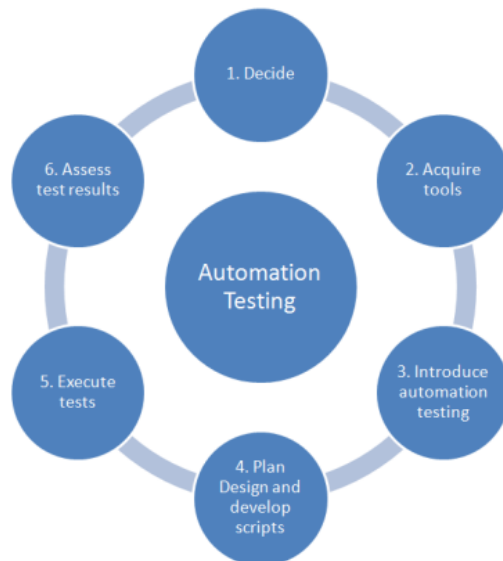
23. Objasnite postupak ručnog testiranja programske podrške uključujući ciljeve i vrste testiranja.

Ručno testiranje je tip testiranja kod kojega se ne upotrebljavaju posebni testni alati ili skripte. **U takvo testiranje uglavnom spadaju neke razine i vrste testiranja** kao što su integration testing, system testing i acceptance testing te istraživačko testiranje. **Cilj ručnog testiranja** je osigurati da sustav radi prema zahtjevima koji su prethodno definirani te identificirati funkcionalne i korisničke pogreške.

Sam **postupak ručnog testiranja** podijeljen je u korake. Prvi korak podrazumijeva analizu zahtjeva, nakon koje se razvija testni plan. Nakon analize zahtjeva i stvaranje plana testiranja, pripremaju se testni slučajevi i kreiraju se testni scenariji i koraci za provjeru funkcionalnosti. Kada su obavljeni svi prethodni koraci, može se prijeći na

izvršavanje testnih slučajeva i to ručno, dakle, tester unosi podatke te prati izlaze i bilježi rezultate. Nakon zabilježenih defekata iste je potrebno ispraviti (obično to rade developeri) te je nakon toga potrebno napraviti verifikaciju da su greške ispravljene.

24. Objasnite i skicirajte postupak automatiziranog/automatskog testiranja programske podrške.



Automatsko testiranje vrsta je testiranja koja uključuje korištenje specijaliziranih alata i skripti za pokretanje testova, usporedbu rezultata i generiranje izvješća. Koraci automatskog testiranja prikazani su na skici.

Decide – korak koji se odnosi na odlučivanje o tome koje će se testne slučajeve automatizirati. Automatizirati je potrebno ponavljajuće i dugotrajne testove.

Acquire Tools – korak u kojem se biraju alati za automatizaciju testiranja koji odgovaraju projektu čije se testove žele automatizirati. Alati se biraju na temelju funkcionalnosti, kompatibilnosti s tehnologijom aplikacije i troškova. Obično se koriste alati kao što su Selenium, JUnit, Appium...

Introduce automation testing – postaviti proces automatizacije unutar organizacije, postavljanje standarda za izradu testnih skripti, kreiranje osnovne testne infrastrukture.

Plan, Design and develop scripts – izrađivanje strategije za automatizaciju i programiranje testnih skripti. Razvija se detaljan plan za pisanje testnih slučajeva (što, kada i kako), pisanje modularnih i ponovno upotrebljivih skripti, definiraju se ulazni podaci i očekivani izlazi.

Execute tests – korak koji se odnosi na izvršavanje testnih skripti. Za njihovo izvršavanje potrebno je osigurati ispravno testno okruženje te je potrebno prikupiti i pratiti rezultate testiranja.

Assess Test Results – korak u kojemu se analiziraju rezultati testiranja kako bi se identificirale pogreške i trendovi. Provjerava se jesu li izlazi testova u skladu s očekivanjima, kategoriziraju se pogreške i prijavljuju razvojnim timovima te se za kraj dokumentiraju rezultati i generiraju se izvješća.

25. Objasnite postupak istraživačkog testiranja programske podrške.

Istraživačko testiranje sastavljeno je od testnih procedura koje se ne prate striktno, a tijekom testiranja otkrivaju se alternativni tokovi upotrebe sustava i odmah se provjeravaju. Iako otkriva mnoge probleme i scenarije koji unaprijed mogu biti planirani, ne vodi se evidencija testiranja, pa uvijek postoje funkcionalnosti koje nisu testirane. **Cilj**

ovakvog testiranja je otkriti neočekivane pogreške koje strukturirani pristup ne bi mogao otkriti, a za takvo što potrebno je iskustvo testera.

26. Objasnite postupak regresijskog testiranja programske podrške.

Regresijsko testiranje metoda je testiranja koja se provodi u iterativnim metodama razvoja programske podrške gdje se tijekom svake iteracije implementira određena funkcionalnost sustava kojoj je testiranje i usmjereno. Rizik koji donosi je što neke funkcionalnosti testirane ranije mogu prestati raditi, jer se tijekom trenutne iteracije promijenio određeni dio koji utječe na ranije implementiranu funkcionalnost, a takve probleme otkriva regresijsko testiranje. Dva su načina za regresijsko testiranje. Potpuno regresijsko testiranje je način gdje se u svakoj iteraciji testiraju sve funkcionalnosti prethodno implementirane. Drugi je način testiranje zavisnih funkcionalnosti, gdje se u svakoj iteraciji testiraju samo funkcionalnosti izvedene u prethodnim iteracijama koje imaju dodirnih točaka s funkcionalnošću u trenutnoj iteraciji.

27. Objasnite black box, white box i gray box metode testiranja programske podrške, te ih međusobno usporedite.

Kriterij	White Box Testing	Black Box Testing	Gray Box Testing
Definicija	Testiranje koje se temelji na unutarnjoj strukturi koda i logici.	Testiranje koje se fokusira na funkcionalnosti sustava bez uvida u kod.	Kombinira znanja o unutarnjoj strukturi i funkcionalnostima sustava.
Pristup	Poznavanje implementacije koda je obavezno.	Nema nikakvog uvida u kod. Fokus na ulazima i izlazima.	Djelomično poznavanje unutarnje logike sustava.
Primjeri	Testiranje petlji, testiranje uvjetnog grananja	Testiranje korisničkog sučelja, funkcionalnosti sustava.	Sigurnosno testiranje kombiniranjem funkcionalnosti i koda.
Prednosti	Omogućuje otkrivanje skrivenih grešaka u kodu. Osigurava potpunu pokrivenost koda.	Fokus na funkcionalnosti sustava. Pogodno za krajnje korisnike.	Kombinira prednosti oba pristupa. Povećava šanse za otkrivanje složenih pogrešaka.
Mane	Zahtijeva tehničko znanje i poznavanje koda. Vrijeme i resursi za analizu.	Ne otkriva greške u strukturi koda ili logici. Oslanja se na testne slučajeve i specifikacije.	Složeniji i zahtjevniji od ostalih pristupa. Potrebna djelomična stručnost u kodu i funkcionalnosti.
Primjena	Unit testing Sigurnosno testiranje na razini koda.	Acceptance testing Usability testing	Sigurnosna i integracijska testiranja.
Alati	JUnit, NUnit, SonarQube.	Selenium, Katalon, Postman.	Burp Suite, OWASP ZAP, SoapUI.
Kompleksnost	Visoka (zahtijeva dubinsko znanje koda).	Niska do srednja (prilagođena korisnicima).	Srednja (kombinira tehničko znanje i funkcionalnost).

28. Objasnite što je funkcionalno testiranje programske podrške, te navedite glavne oblike funkcionalnog testiranja.

Funkcionalno je testiranje takav oblik testiranja u kojem se provjerava zadovoljava li sustav prethodno postavljene funkcionalnosti koje su definirane u specifikacijama, pri čemu se vrši pregled zahtjeva i prolazi kroz različite scenarije korištenja sustava te se utvrđuje ima li propusta. Cilj je provjeriti izvršava li sustav sve što je potrebno i izvršava li sustav nešto što ne treba. **Da bismo ispitali funkcionalnosti neke aplikacije** prvo je potrebno odrediti funkcionalnosti koje aplikacija treba omogućiti, stvoriti skup podataka na temelju specifikacija aplikacije, izlaz treba biti temeljen na testnim podacima i specifikacijama aplikacije, potom pišemo testne scenarije i izvodimo testne slučajeve te na kraju uspoređujemo postignute i očekivane rezultate.

Glavni oblici funkcionalnog testiranja su testiranje jedinica (unit testing), integracijsko testiranje (integration testing), testiranje sustava (system testing), regresijsko testiranje (regression testing), testiranje prihvatanja (acceptance testing).

29. Objasnite što je i navedite obilježja testiranje cjelina/jedinica koda (unit testing).

Unit testiranje je testiranje koje uglavnom pokreću programeri prije prosljeđivanja testnom timu koji formalno pokreću testne slučajeve. Unit testiranje provodi se izolirano na pojedinačnim cjelinama izvornog koda (klase, metode i slično) te predstavljaju najnižu razinu testiranja koda te se zbog toga često i automatizira. **Cilj je kod ovog testiranja** izdvojiti svaki dio programa i pokazati da je svaki od njih ispravan s gledišta zahtjeva i funkcionalnosti. Ograničenja ovakvog testiranja su da se ne može uhvatiti svaka pogreška u programu te ima ograničen broj scenarija i testnih podataka koji se mogu koristiti.

30. Objasnite što je, navedite obilježja i način provedbe integracijskog testiranja programske podrške (integration testing).

Integracijsko testiranje je testiranje dijelova aplikacije, zbirki podsustava ili eventualno cijelog sustava s ciljem utvrđivanja rade li ispravno kao cjelina. Provode ga programeri i testni tim s ciljem testiranja sučelja među podsustavima. Izvodi se kao: **Bottom-up integracija** – testiranje počinje s unit testiranjem nakon kojega slijedi testiranje kombinacija modula. Uobičajeno se izvodi prvo. **Top-down integracija** – moduli više razine testiraju se prvi, a poslije njih moduli nižih razina. Uobičajeno se izvodi nakon bottom-upa.

31. Objasnite što je, navedite obilježja i način provedbe testiranja programske podrške na razini sustava (system testing).

Testiranje sustava je testiranja cijelog sustava sa svim integriranim komponentama, uglavnom od strane posebnog testnog tima. Testiranje sustava predstavlja prvo testiranje cijelog sustava u ciklusu razvoja, ono verificira zadovoljava li aplikacija funkcijske i tehničke specifikacije. Testiranje se provodi u okolini koja je vrlo bliska produkcijskoj te je omogućeno testiranje poslovnih zahtjeva i arhitekture aplikacije.

Da bismo testirali sustav, prethodno je potrebno obaviti integracijsko testiranje. Potrebno je imati kreiran testni plan i na temelju njega stvoriti testne slučajeve i testne podatke za izvođenje testova. Zatim se izvršavaju testni slučajevi i na kraju se prijavljuju i ispravljaju pogreške.

32. Usporedite integracijsko testiranje i testiranje sustava.

Karakteristika	Integracijsko testiranje	Testiranje sustava
Opseg testiranja	Testira integraciju među modulima kako bi se	Testira cijeli sustav sa svim integriranim komponentama.

	<i>provjerilo rade li ispravno zajedno kao grupa.</i>	
Vrste testiranja	<i>Fokusira se na funkcionalno testiranje.</i>	<i>Obuhvaća funkcionalno i nefunkcionalno testiranje (performanse, sigurnost, stres itd.).</i>
Preduvjeti	<i>Provodi se nakon što su moduli prošli unit testiranje.</i>	<i>Provodi se nakon uspješnog integracijskog testiranja.</i>
Početna točka	<i>Počinja od specifikacije sučelja.</i>	<i>Počinja od zahtjeva i specifikacija sustava.</i>
Metode testiranja	<i>Kombinacija white-box i black-box testiranja.</i>	<i>Black-box testiranje.</i>
Fokus testiranja	<i>Fokusira se na provjeru integracije među modulima.</i>	<i>Fokusira se na funkcionalnost i ponašanje sustava kao cjeline.</i>
Cilj	<i>Osigurati ispravno funkcioniranje modula u kombinaciji.</i>	<i>Potvrditi da sustav zadovoljava poslovne i tehničke zahtjeve.</i>

33. Objasnite što je, navedite obilježja i način provedbe regresijskog testiranja programske podrške (regression testing).

Regresijsko je testiranje vrsta testiranja koja služi da promjene, poput novih značajki ili ispravljenih pogrešaka, nisu negativno utjecale na postojeću funkcionalnost sustava koja ispravno radi. S obzirom da se sustav često proširuje ili mijenja, potrebno je često iznova testirati već testirane funkcionalnosti, pa se zbog toga često automatiziraju takvi testovi. Regresijsko testiranje se provodi na način da se prvo identificiraju promjene u sustavu, nakon čega se odaberu testovi za regresijsko testiranje. Takvi su testovi uglavnom vezani uz funkcionalnost koja je pogođena s novom izmjenom u sustavu. Kako bi testiranje bilo učinkovitije, automatiziramo ga te izvedemo testove. Nakon izvedenih testova se analiziraju rezultati testova, prijavljuju i ispravljaju pogreške te se zatvara ciklus regresijskog testiranja. Prednost regresijskog testiranja je trenutno uočavanje pogrešaka, smanjenje nepotrebnih troškova, a mana je da se regresijsko testiranje treba izvesti nakon promjene koda, ali se često testiranje ne može izvesti brzo, pa razvojni proces može kasniti.

34. Objasnite što je, navedite obilježja i način provedbe testiranja prihvaćanja programske podrške (acceptance testing).

Testiranje prihvaćanja posljednja je faza u testiranju softvera te osigurava da je softver spreman za upotrebu od strane krajnjih korisnika. Testiranje prihvaćanja utvrđuje koliko aplikacija ispunjava zahtijevane specifikacije i zadovoljava zahtjeve klijenata te potvrđuje je li sustav spreman za isporuku. Testiranje se provodi u dvije faze. **Alpha testing** – prva faza testiranja i izvodi se između timova, podrazumijeva unit, integracijsko i testiranje sustava. Otkrivaju se pogreške u pisanju, neispravne poveznice i ponašanje na sustavu najnižih dopuštenih performansi. **Beta testing** – obavlja se poslije alpha testiranja. U ovoj fazi šira skupina testira instaliranje, pokretanje, neispravnosti u slijedu izvođenja, ispade sustava te se ispituje rad na stvarnim problemima korisnika.

35. Objasnite što je nefunkcionalno testiranje programske podrške, te navedite glavne oblike nefunkcionalnog testiranja.

Nefunkcionalno je testiranje vrsta testiranja koja se koristi za provjeru nefunkcionalnih zahtjeva sustava definiranih u specifikacijama. Primjer nekih nefunkcionalnih zahtjeva na

sustav bi bili performanse, upotrebljivost, pouzdanost, sigurnost, dostupnost, skalabilnost i slično. Nefunkcionalno testiranje treba biti mjerljivo, važno je odrediti prioritet zahtjeva te provjeriti jesu li pokazatelji kvalitete ispravno identificirani. Ciljevi nefunkcionalnog testiranja su povećati učinkovitost i održivost sustava, smanjenje rizika proizvodnje i troškove, optimirati način rada proizvoda.

Glavni oblici nefunkcionalnog testiranja su: testiranje performansi (performance testing), testiranje upotrebljivosti (usability testing) i testiranje sigurnosti (security testing).

36. Objasnite što je, navedite obilježja i oblike provedbe testiranja performansi programske podrške (performance testing).

Testiranje performansi proces je koji se provodi kako bi se utvrdilo hoće li određeni proizvod dobro izvesti očekivane procese pod različitim radnim opterećenjima. Konkretno se odnosi na pronalaženje uskih grla sustava kao što su kašnjenje mreže, problemi s obradom na strani klijenta, neuravnoteženosti opterećenja servera. Parametri za testiranje performansi su brzina, kapacitet, stabilnost i skalabilnost. Testiranje može biti **kvalitativno** (subjektivni aspekti u procjeni performansi sustava pri nekom opterećenju) i **kvantitativno** (mjerenje performansi kroz brojčane vrijednosti), a dijeli se na testiranje opterećenja i stres test. **Testiranje opterećenja** odnosi se na prosječno opterećenje podacima, dok se **stres testiranje** odnosi na testiranje programa u neuobičajenim uvjetima (preopterećenje, isključivanje baze i slično).

37. Ukratko objasnite što je propusnost, a što devijacija kod testiranja performansi.

Propusnost i devijacija dva su parametra koja se koriste za mjerenje performansi sustava.

Propusnost označava količinu podataka ili broj operacija koje sustav može obraditi u određenom vremenskom razdoblju. Propusnost predstavlja mjeru efikasnosti i kapaciteta sustava koja se obično izražava u jedinicama kao što su bitovi po sekundi, paketi po sekundi, transakcije po sekundi itd. Primjer: Ako neki server može obraditi 1000 zahtjeva u sekundi, propusnost servera je 1000 zahtjeva po sekundi.

Devijacija se odnosi na razlike u vremenu odziva ili performansama sustava pod različitim uvjetima. Predstavlja mjeru nestabilnosti sustava i pokazuje koliko su rezultati testiranja udaljeni od prosječne vrijednosti. Primjer: Ako sustav u prosjeku odgovara za 2 sekunde, ali ako je sustav preopterećen odgovarat će za 5 sekundi, pa će devijacija biti visoka, a visoka devijacija ukazuje na nestabilnost performansi.

38. Objasnite što je, navedite obilježja i oblike provedbe testiranja upotrebljivosti programske podrške (usability testing).

Testiranje upotrebljivosti je black-box tehnika testiranja koja se provodi kako bi se identificirale greške, poboljšanja i problemi u aplikaciji gledajući kako korisnici koriste sustav. Cilj je osigurati da korisničko sučelje i funkcionalnosti aplikacije budu učinkoviti, da sučelje bude intuitivno te da u konačnici korisnici budu zadovoljni tijekom interakcije s programom. Gledajući korisnike kako koriste sustav, mjeri se njihova učinkovitost korištenja, mogućnost učenja, pamćenje korištenja te pogreške u radu. Primjer oblika testiranja su **testiranje pomoću pravih korisnika** i **A/B testiranje** (usporedba dvije različite verzije sučelja),

39. Objasnite što je, navedite obilježja i oblike provedbe testiranja sigurnosti programske podrške (security testing).

Testiranje sigurnosti proces je provjere sigurnosnih mjera i zaštite softvera kako bi se osiguralo da sustav štiti podatke i resurse od prijetnji te radi u skladu sa sigurnosnim pravilima i zahtjevima definiranim u zahtjevima sustava. Cilj je identificirati slabosti i

ranjivosti koje bi mogle biti iskorištene od strane napadača. **Glavna obilježja** su povjerljivost (samo ovlašteni korisnici mogu pristupiti osjetljivim podacima), cjelovitost (očuvanje točnosti podataka), autentikacija (provjera metoda autentikacije), neodricanje, sigurnost podataka i slično. **Oblici provedbe testiranja sigurnosti** uključuju provjeru ulaza (testiranje kako bi se osiguralo da sustav pravilno obrađuje sve unose podataka), SQL Insertion napadi (testiranje kako bi se otkrile ranjivosti koje omogućuju umetanje zlonamjernih SQL naredbi), injektiranje nedostataka (namjerno unošenje pogrešaka kako bi se testirala otpornost sustava), upravljanje sesijama, skriptni napadi i prepunjavanje spremnika.

40. Objasnite što je, navedite obilježja i prednosti razvoja programske podrške pokretanog testiranjem (test-driven development).

Razvoj pogonjen testiranjem razvoj je programa koji se oslanja na ponavljanje vrlo kratkog razvojnog ciklusa, koji uključuje pisanje testnih slučajeva usporedno s pisanjem koda, što uvelike poboljšava razvoj. Takav razvoj započinje s projektiranjem i razvojem testova za svaku malu funkcionalnost aplikacije. Prvo se razvija test koji određuje i provjerava što će kod učiniti, a zatim se piše kod koji mora proći sve postavljene testove. Ciklus se sastoji od 3 faze. Crvena faza – pišu se testni slučajevi za postupke koji će biti provedeni testiranjem. Zelena faza – faza u kojoj se piše samo programski kod. Faza refaktoriranja – izmjena koda, tako da svi testovi dođu u zelenu fazu uz obvezno uklanjanje dupliciranog koda.

Prednosti ovakvog pristupa u razvoju su rana obavijest o pogreškama, bolji dizajn i čišći kod, izvođenje provjera u fazi refaktoriranja, dobar timski rad, osigurava da sustav zadovolji definirane zahtjeve uz veće povjerenje zbog testiranja svih dijelova koda, veća pokrivenost koda.

41. Objasnite korake i faze ciklusa razvoja programske podrške pokretanog testiranjem. Koraci razvoja programske podrške pokretanog testiranjem:

Prvi korak u razvoj pokretanog testiranjem je pisanje i dodavanje testa. Ovaj test u početku ne prolazi, jer funkcionalnost još uvijek nije implementirana. Nakon pada testa, potrebno je napisati minimalnu količinu koda kako bi test prošao. Nakon toga, ponovno se izvršava testiranje i ako test prođe proces se vraća na početak za dodavanje novog testa, a ako test vrši se refaktoriranje koda te se nakon toga opet vrši testiranje. Nakon što se sve refaktorira i testira, može se prijeći na pisanje testa za novu funkcionalnost.

Faze ciklusa razvoja programske podrške pokretanog testiranjem:

Ciklus se sastoji od 3 faze. Crvena faza – pišu se testni slučajevi za postupke koji će biti provedeni testiranjem. Zelena faza – faza u kojoj se piše samo programski kod. Faza refaktoriranja – izmjena koda, tako da svi testovi dođu u zelenu fazu uz obvezno uklanjanje dupliciranog koda.

42. Objasnite razliku između statičkog i dinamičkog testiranja programske podrške, te navedite najčešće oblike navedenih načina testiranja.

Statičko se testiranje provodi prije izvođenja koda analizirajući kodove i dokumentaciju kako bi se pronašle pogreške i propusti. **Cilj** takve vrste testiranja je identificirati probleme u ranim fazama razvoja čime se smanjuju troškovi ispravljanja pogrešaka.

Prednost takvog načina je otkrivanje problema bez potrebe za izvršavanjem koda, smanjenje troškova otklanjanja grešaka i veća kvaliteta koda. **Glavne metode** takvog testiranja su statička analiza (korištenje alata za automatsko otkrivanje problema poput neispravne sintakse), pregled (rukovoditelji ili stručnjaci analiziraju dokumente i kod), neformalni prolazak kroz kod (uglavnom autor koda prođe kroz vlastiti kod tražeći

pogreške, uglavnom bez smjernica) i formalna inspekcija koda (detaljan prolazak kroz kod uz utvrđene smjernice).

Dinamičko se testiranje provodi za vrijeme izvođenja koda te se utvrđuje radi li sustav prema očekivanjima. **Cilj** ovakvog pristupa je provjeriti funkcionalnost, performanse, sigurnost i ponašanje softvera u različitim uvjetima i to na način da se testiranje vrši kroz interakciju s programom dok je on u pokretu uglavnom pomoću nekih ulaznih podataka koji izazivaju odgovarajuće izlaze. **Prednosti** ovakvog pristupa su mogućnost da se validiraju funkcionalnosti, identificiraju probleme s performansama te mogućnost testiranja interakcija među komponentama. **Glavne metode** takvog testiranja su black-box i white-box testiranje.

43. Ukratko objasnite što je i kako se izvodi statička analiza programskog koda.

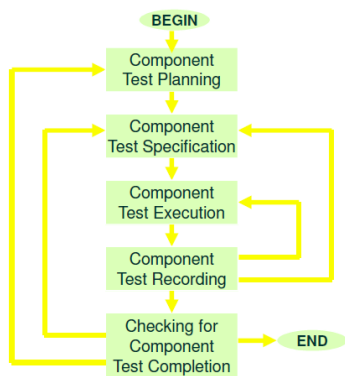
Statička analiza programskog koda tehnika je koja podrazumijeva pregled i analizu sustava bez njegovog izvršavanja. Cilj je identificirati pogreške, sigurnosne ranjivosti i druge nedostatke što ranije tijekom razvoja softvera.

Izvođenje se vrši tako da se odabere alat za statičku analizu te se postave pravila i smjernice za analizu (standardi kodiranja, sigurnosne smjernice i slično). Zatim se pokreće alat za analizu koji pregledava kod liniju po liniju, a iz koda se tada generiraju različiti modeli kao što su apstraktno sintaksko stablo, graf toka kontrole i graf poziva funkcija. Nakon ekstrakcije modela, alat generira nekoliko reprezentacija kako bi olakšao analizu koda, njegove strukture, toka i odnosa među metodama. Nakon toga, alat analizira kod i kreira detaljan izvještaj koji uključuje kategorije problema (kritična greška, upozorenje, preporuka), lokaciju problema u kodu te upute za ispravak. Nakon generiranja izvješća, ono se pregledava i korigira.

44. Ukratko objasnite heuristiku koja se primjenjuje kod testiranja jedinica koda.

Kreirati unit testove nakon što je dizajn objekta kompletan. Black-box testiranje se koristi za testiranje funkcionalnog modela jedinice s fokusom na ulaz i izlaz. Dok se white-box testiranje koristi za testiranje dinamičkog modela jedinice, analizirajući unutrašnju strukturu koda. **Razvoj testnih slučajeva** (potrebno je pronaći efektivan broj testnih slučajeva). **Provjera testnih slučajeva s ciljem eliminiranja duplikata** (testni slučajevi se pregledavaju kako bi se eliminirali duplikati i izbjeglo gubljenje vremena na redundantne testove). **Provjera izvornog koda** ponekada štedi vrijeme jer omogućava rano otkrivanje potencijalnih grešaka prije pokretanja testova. **Kreiranje testne cjeline** podrazumijeva izradu testnih pogonskih programa i testnih izdanaka koji su nužni za integracijsko testiranje. **Opis testnog predviđanja** opisuje očekivane rezultate testova i često se generira nakon prvog uspješnog testiranja. **Izvođenje testnih slučajeva** podrazumijeva pokretanje testova i njihovo ponavljanje nakon svake promjene koda, što je poznato kao regresijsko testiranje. **Usporedba rezultata testa s predviđanjem** postupak je koji bi trebalo automatizirati ako je to moguće.

45. Skicirajte i ukatko objasnite dijagram postupka testiranja jedinica koda.



Planiranje testiranja komponenti (component test planning) – prva faza u postupku testiranja jedinica koda, koja uključuje definiranje kako će se strategija testiranja primijeniti na određenu komponentu. Identificiraju se svi izuzetci i posebni uvjeti koji se primjenjuju na neku komponentu. Definiraju se svi softveri s kojima će komponenta koja se testira komunicirati tijekom testiranja.

Specifikacija testiranja komponenti (component test specification) – druga faza, odnosi se na dizajniranje testnih slučajeva koristeći tehnike navedene u testnom planu. Testni slučaj mora imati cilj testa, početno stanje komponenti, ulazne podatke te očekivani ishod. Testni bi slučajevi trebali biti ponovljivi kako bi se mogli više puta izvoditi.

Izvršenje testiranja komponenti (component test execution) – faza u kojoj se izvršavaju testovi. Standardi ne određuju kako će se testovi izvoditi, ali je bitno da svi testovi koji su planirani budu provedeni.

Zapisivanje rezultata testiranja komponenti (component test recording) – faza u kojoj se bilježe podaci o komponentama, verzijama, stvarnom ishodu testa i njegovoj usporedbi s očekivanim ishodom. Ako postoji neslaganje, ono se bilježi i test se ponovno pokreće kako bi se potvrdilo uklanjanje grešaka. Razina pokrivenosti testa se bilježi i cilju ocjene je li testiranje dovršeno prema kriterijima iz testnog plana.

Provjera dovršenosti testiranja komponenti (checking for component test completion) – posljednja faza u postupku unit testiranja, provodi se provjera o tome jesu li svi testovi završeni u skladu s definiranim kriterijima za testiranje. Ako kriteriji nisu zadovoljeni, testne se aktivnosti ponovno izvode.

46. Objasnite postupak izmjene programskog koda (refactoring), te njegovu povezanost s testiranjem jedinica koda.

Refaktoriranje koda proces je inkrementalnog poboljšavanja postojećeg koda i pretpostavlja da se iz prvog pokušaja ne napravi najbolje rješenje. Sloboda u refaktoriranju u prvom je redu omogućena unit testiranjem koje je osnova za sigurno refaktoriranje i s kojim treba što prije početi, tako da se svakom promjenom može pronaći pogreška i to spiralnim pristupom, kojim se povećava kvaliteta i istodobno dodaju nove značajke. Refaktoriranje se često provodi uz podršku metoda poput Arrange, Act, Assert (AAA) ili Red, Green, Refacto (uglavnom u test-driven razvoju).

47. Objasnite što je AAA (Arrange, Act, Assert) uzorak testiranja jedinica koda i na kratkom primjeru objasnite provedbu takvog testiranja.

Arrange, Act, Assert uzorak testiranja je struktura koja pomaže organizirati testove, čineći ih čitljivima i razumljivima. Sastoje se od tri koraka: **Arrange** (dogovaranje preduvjeta testa): faza postavljanja okruženja za testiranje. Uključuje stvaranje objekata potrebnih za testiranje, određivanje testnih scenarija, definiranje početnog stanja i pripremu

ulaznih podataka. **Act** (izvođenje): faza u kojoj se izvodi radnja koja se testira (npr. poziv metode). **Assert** (potvrda da se dogodilo očekivano ponašanje): posljednja faza u kojoj se provjerava je li rezultat radnje u skladu s očekivanim ishodom uspoređujući stvarne i očekivane vrijednosti.

Primjer:

```
[TestMethod]
public void AddingItemIncreasesCartCount()
{
    // Arrange
    var cart = new ShoppingCart();
    var item = new Item { Name = "Laptop", Price = 1000.00m };

    // Act
    cart.AddItem(item);

    // Assert
    Assert.AreEqual(1, cart.ItemCount, "Broj stavki u košarici trebao bi biti 1 nakon dodavanja.");
}
```

Arrange: prije testiranja, potrebno je definirati scenarij. Ovdje testiramo klasu *Shopping card* koja ima metodu *AddItem* za dodavanje stavki u košaricu. Potrebno je pripremiti sustav za testiranje kreiranje instance klase *ShoppingCart* i definirati ulazne testne podatke. U ovom primjeru, stvara se instanca s praznom košaricom i kreira se stavka koja će se u sljedećoj fazi dodati u košaricu.

Act: Nakon što smo inicijalizirali košaricu i definirali stavku koju ćemo dodati, pozivamo metodu *AddItem* s argumentom *item*.

Assert: Provjerava se očekivano ponašanje sustava nakon izvršene akcije. Očekujemo da će se broj stavki povećati za 1 i koristimo metodu *Assert.AreEqual* za usporedbu stvarne i očekivane vrijednosti.

48. Objasnite na primjeru provedbu razvoja pokretanog testiranjem (Red, Green, Refactor) kroz testiranje jedinica koda.

Razvoj pokretan testiranjem temelji se na pisanju testova prije samog implementiranja funkcionalnosti, a taj proces prolazi kroz tri faze: *Red*, *Green* i *Refactor*. Prvo se piše neuspješni test (**Red**), zatim se implementira minimum koda da test uspije (**Green**), a zatim se refaktorira i poboljšava kod uz zadržavanje ispravnog ponašanja (**Refactor**).

Primjer:

```

public class ShoppingCartTests {

    [TestMethod]
    public void AddingItemIncreasesCartCount()
    {
        // Arrange
        var cart = new ShoppingCart();
        var item = new Item { Name = "Laptop", Price = 1000.00m };

        // Act
        cart.AddItem(item);

        // Assert
        Assert.AreEqual(1, cart.ItemCount, "Broj stavki u košarici trebao bi biti 1 nakon dodavanja.");
    }

}

```

RED: Pišemo testove (kao kod AAA) te ih pokrenemo. Testovi bi trebali pasti zbog toga što nismo implementirali jedinice koje testovi testiraju.

```

public class Item
{
    public string Name { get; set; }
    public decimal Price { get; set; }
}

public class ShoppingCart
{
    private int itemCount = 0;

    public int ItemCount => itemCount;

    public void AddItem (Item item)
    {
        itemCount++;
    }
}

```

GREEN: U ovom koraku pišemo minimalnu količinu koda, tako da testovi ne padnu. U ovom koraku dozvoljeno je kršiti neka načela razvoja programske podrške.

```
public class ShoppingCart
{
    private readonly List<Item> items = new List<Item>();

    public int ItemCount => items.Count;

    public void AddItem(Item item)
    {
        items.Add(item);
    }
}
```

REFACTOR: U ovom koraku unaprjeđujemo kod te ga čistimo od dupliciranja koda i sličnih loših načina dizajniranja koji su nastali uslijed potrebe za zadovoljavanjem testova. Testove se ne refaktorira, jer se na testovima temelji razvoj programske podrške, a eventualnom promjenom testova srušili bi se temelji razvoja (to više ne bi bilo test-driven).

Nakon refaktoriranja, potrebno je još jednom testirati izmjene te ako testovi prođu, završili smo za testiranjem te jedinice.

49. Navedite najčešće zahvate izmjene programskog koda (refactoringa).

Zamjena magičnih brojeva konstantom, zamjena uvjetnih izraza polimorfizmom, zamjena konstruktora metodom tvornice, promjena imena metoda, klasa, modula i sličnog, uklanjanje mrtvog koda, uklanjanje dupliciranog koda...

50. Navedite i ukratko opišite načine korištenja testiranja jedinica koda za poboljšanje programskog koda.

Poboljšanje pouzdanosti koda (unit testovi osiguravaju da pojedinačne jedinice ispravno rade, kada su testovi prisutni, greške se lakše identificiraju i otklanjaju), **veća održivost koda** (kada imamo napisane unit testove, dodavanje novih funkcionalnosti u kod ne bi trebalo predstavljati problem, s obzirom da ako pokvarimo postojeću funkcionalnost koda, testovi će to pokazati), **sigurno refaktoriranje** (testovi potvrđuju da promjene na kodu nisu narušile očekivano ponašanje koda), **modularan dizajn** (s obzirom da se testiraju zasebne jedinice koda, kod se piše i testira po jedinicama te se postiže modularan dizajn), **brže otkrivanje grešaka** (greške u implementaciji se otkrivaju brže, jer postoje testovi koji ukazuju na njih), **osiguravanje regresijskog testiranja** (nakon svake promjene koda možemo pokrenuti unit testove i time osigurati da novonastali kod nije narušio prethodno napisan kod koji je ispravno radio), **otkrivanje nedostataka u rubnim slučajevima...**

51. Objasnite što je, navedite obilježja i oblike provedbe testiranja prenosivosti programske podrške (portability testing).

Testiranje prenosivosti ili portability testing odnosi se na proces ocjenjivanja sposobnosti programske podrške da ispravno radi u različitim okruženjima bez potrebe za značajnim promjenama. **Cilj ovakvog testiranja** je da aplikacija može uspješno raditi neovisno o platformi, operacijskom sustavu, hardveru i sličnim okruženjima. **Obilježja ovakvog testiranja** su da se testira kompatibilnost aplikacije s različitim operacijskim sustavima, procjenjuje se koliko su promjene potrebne za prilagodbu aplikacije u drugom okruženju, testira se slijedi li aplikacija standarde i norme te se testira ovisnost aplikacije o specifičnim hardverskim i softverskim komponentama.

Oblici testiranja su testiranje na različitim platformama, testiranje migracije, testiranje na različitim konfiguracijama, testiranje kompatibilnosti s različitim verzijama softvera i testiranje mrežne prenosivosti (ispituje se sposobnost aplikacije da se prilagodi različitim mrežnim uvjetima i protokolima).

52. Objasnite testiranje kompatibilnosti (backward compatibility testing).

Testiranje kompatibilnosti ili backward compatibility testing odnosi se na proces provjere funkcionira li novo razvijeni ili ažurirani softver ispravno s prethodnim verzijama okruženja, uključujući formate datoteka, tablica podataka i struktura podataka koje su stvorene u starijim verzijama softvera. Ovim testiranjem osigurava se da korisnici koji prelaze na novu verziju softvera mogu i dalje koristiti postojeće podatke i formate bez problema s kompatibilnošću.

53. Objasnite pojam comparison testing, te navedite i objasnite dva tipa takvog testiranja.

Comparison testing uspoređuje dobre i loše strane nekog proizvoda u odnosu na njegove prethodne verzije ili slične proizvode. Postoje dva tipa takvog testiranja, a to su direktna i objektivna komparacija.

Direct comparison ili direktna komparacija odnosi se na tip testiranja gdje više proizvoda testiramo istovremeno, dok se u **objective comparison** ili **objektnoj komparaciji** proizvodi testiraju jedan po jedan istim testovima.

54. Objasnite testiranje usklađenosti (compliance testing).

Testiranje usklađenosti ili compliance testing odnosi se na testiranje softvera s obzirom na unutarnje i vanjske standarde koje mora slijediti. Unutarnji standardi odnose se na skup standarda koje određuje kompanija i odlučuje provoditi ih u softveru kojeg se testira. S druge strane, vanjski standardi bili bi oni koje softver mora slijediti zbog raznih razloga (npr. zakonske regulative).

55. Objasnite pojam gorilla testing.

Gorilla testing je vrsta testiranja u kojoj se intenzivno i detaljno ispituje jedan modul ili određena funkcionalnost unutar modula. Testiranje provodi tester, a ponekad i sam programer. Modul se testira više puta (nerijetko i stotinama puta) na isti način kako bi se osiguralo da modul radi ispravno i da nema pogrešaka. Cilj gorilla testinga jest provjeriti robusnost aplikacije.

56. Objasnite mutacijsko testiranje (mutation testing).

Mutacijsko testiranje ili mutation testing vrsta je white-box testiranja u kojoj se kod namjerno mijenja s ciljem provjere hoće li testovi identificirati takve defekte u sustavu. Promjene u izvornom kodu su minimalne tako da ne utječe na cijelu aplikaciju, nego samo na specifičan dio aplikacije.

57. Objasnite pojam recovery testing.

Recovery testing je proces testiranja kojim se procjenjuje sposobnost aplikacije ili sustava da se oporavi nakon nepredviđenih kvarova ili katastrofa. Cilj ovog testiranja je

provjeriti može li se sustav vratiti u operativno stanje nakon prekida te nastaviti raditi s minimalnim gubitkom podataka ili funkcionalnosti.

Primjer takvog testiranja uključuje simulaciju situacije u kojoj je aplikacija povezana na mrežu, ali dolazi do prekida, poput odspajanja mrežnog kabela. Nakon ponovnog uspostavljanja veze, testira se može li sustav nastaviti primati podatke točno od trenutka prekida.

58. Objasnite pojam sanity testing.

Sanity testing je vrsta testiranja koja se provodi kako bi se utvrdilo je li nova verzija softvera (s naglaskom na nove funkcionalnosti softvera) dovoljno stabilna za daljnje, opsežnije testiranje. Ovaj test fokusira se na specifične komponente sustava koje su promijenjene ili koje sadrže novu funkcionalnost i podskup je regresijskog testiranja. Ako se aplikacija odmah nakon početnog pokretanja počne rušiti, to znači da sustav nije dovoljno stabilan za daljnje testiranje. U takvim slučajevima, aplikacija ili verzija softvera se vraća na popravak grešaka prije nego što se nastavi s opsežnijim testiranjem. Sanity testing uglavnom izvode tester i najčešće nije dokumentiran.

59. Objasnite pojam smoke testing.

Smoke testing je vrsta testiranja softvera koja uključuje osnovni skup testova s ciljem osiguravanja da najvažnije funkcionalnosti softvera ispravno rade i da softver ne sadrži ozbiljne greške. Smoke testing pokriva cijeli sustav i podskup je acceptance testinga. Rezultati ovog testiranja koriste se za odlučivanje je li nova verzija softvera dovoljno stabilna za daljnje, detaljnije testiranje. Ako aplikacija pokazuje ozbiljne greške ili nije stabilna, testiranje se obustavlja dok se problemi ne isprave. Smoke testing uglavnom izvode developeri ili tester i te je najčešće dokumentiran.

60. Ukratko objasnite pojmove: JSON, REST, REST web-usluga, REST API.

JSON (JavaScript Object Notation) transportni je format podataka koji je jednostavan, lagan za čitanje i parsiranje. Koristi se za prijenos podataka između klijenata i servera i poznat je kao standard u mnogim web aplikacijama.

Primjer: {
 "ime": "Marko",
 "dob": 25,
 "adresa": {
 "ulica": "Zagrebačka 1",
 "grad": "Zagreb"
 }
}

REST (Representational State Transfer) jedan je od načina arhitekture softvera na WWW-u. REST je koordinirani skup arhitekturnih ograničenja na komponente, sudionike konekcije i elemente podataka unutar distribuiranog sustava. Arhitekturna ograničenja REST-a su da se klijenti ne brinu za spremanje podataka, a server se ne brine za korisničko sučelje, server ne pamti nikakva stanja klijenat, mogućnost priručne memorije, klijent nema znanje je li spojen na međuserver ili na krajnji server te najvažnije ograničenje – jedinstveno sučelje.

REST web usluga je web usluga koja koristi REST arhitekturu za komunikaciju između klijenta i servera. Ove usluge omogućuju aplikacijama razmjenu podataka i resursa putem standardnih HTTP metoda GET, PUT, DELETE, HEAD, POST gdje svaki resurs ima svoj URI, a resursi su često pisani u JSON formatu.

REST API (Application programming interface) sučelje koje omogućava aplikacijama da komuniciraju koristeći REST arhitekturu. API je skup pravila i protokola koji definiraju kako

različite aplikacije mogu međusobno komunicirati. REST API omogućuje klijentima da šalju HTTP zahtjeve prema serveru i obavljaju akcije nad resursima. Uglavnom se koristi JSON format za razmjenu podataka.

61. Objasnite što, kada i kako se testira automatiziranim testiranjem programskog koda.

U programu ili testu se automatiziraju korisničke transakcije kao što su prijava korisnika, istovremeni pristup programu od strane velikog broja korisnika, dijelovi korisničkog sučelja, spajanje s bazom podataka, validiranje polja i slično.

Kada nešto automatski testirati treba odlučiti s timom, a najčešće se automatizira ako su u pitanju veliki i kritični projekti, projekti koji zahtijevaju često testiranje istih dijelova, ako se zahtjevi ne mijenjaju često, pristup aplikaciji za opterećenje i performanse od strane više virtualnih korisnika, program je stabilan s gledišta ručnog testiranja te ovisno o raspoloživosti vremena.

Automatiziranje se obavlja nekim od jezika koji ga podržavaju i automatiziranom programskom aplikacijom. Postoji niz alata koji omogućuju pisanje skripti. Prilikom automatizacije potrebno je identificirati područja programa za automatiziranje, odabrati alat za automatiziranje, pisati testne skripte, razviti testne okoline, izvesti skripte, stvoriti izvješća s rezultatima i identificirati potencijalne pogreške ili probleme s performansama.

62. Navedite najznačajnije alate za automatizirano testiranje programskog koda.

Visual Studio Test Professional, Selenium, Saucelabs, Browsera, Browserstack, Katalon Studio, Appium, Espresso, SilkTest, TestComplete, LoadRunner...

63. Objasnite što je, od čega se sastoji i kako se koristi testni okvir alata Selenium, te što je i kako radi Selenium WebDriver.

Selenium je skup alata koji omogućuju automatizirano testiranje, može simulirati kretanje korisnika kroz stranice i staviti oznake na stranice. Sadrži višestruke funkcije, koje su vrlo prilagodljive, a služe prepoznavanju elemenata korisničkog sučelja i interakciji s njima.

Selenium se sastoji od Selenium WebDrivera (služi za kreiranje robusne, automatizirane na pregledniku zasnovane regresijske okoline i testova, a također skalira i raspodjeljuje skripte među raznim okolinama), **Selenium IDE** (kreiranje brzih skripti za reproduciranje grešaka, kreira skripte koje pomažu automatizirano eksploratorno testiranje), **Selenium Grid** (koristi se za pokretanje paralelnih testova u različitim preglednicima, omogućujući testiranje na različitim platformama istovremeno) i **Selenium Remote Control** (alat koji omogućuje testiranje web aplikacije u različitim preglednicima koristeći server koji kontrolira preglednike i omogućuje izvršavanje testova).

Selenium WebDriver komponenta je unutar Selenium alata, a omogućava kontrolu nad web preglednicima putem API-ja, što omogućuje izravno upravljanje različitim preglednicima. WebDriver se povezuje s različitim preglednicima putem posebnih drivera koji omogućuje posebnu komunikaciju s preglednicima, jer svaki preglednik ima vlastitu implementaciju za automatizaciju. Posebno je koristan za dinamičke web stranice, jer omogućava precizno dohvaćanje elemenata pomoću različitih identifikatora. **Za korištenje Selenium WebDrivera** prvo je potrebno instalirati i postaviti okruženje te pokrenuti WebDriver koji nam omogućava komuniciranje s web preglednicima. Nakon inicijalizacije WebDrivera, može se otvoriti web stranice pomoću metode `get()` koja učitava URL stranicu u pregledniku. Zatim, Selenium omogućuje prepoznavanje i interakciju s HTML elementima na stranici. Na tim elementima moguće je izvesti različite radnje, a nakon izvođenja željenih radnji, izvršavamo validaciju i provjeru rezultata.

64. Nabrojite tipove Selenium lokatora, te navedite koji se od njih najčešće koristi i zašto.

Selenium lokatori su mehanizmi pomoću kojih Selenium prepoznaje i postiže interakciju s elementima na web stranici. **Selenium lokatori su:** **ID lokator** (atribut id HTML elementa), **Name lokator** (atribut name HTML elementa), **Class name lokator** (temeljen na atributu class), **Tag Name lokator** (lokator temeljen na imenu HTML taga), **Link Text lokator** (lokator temeljen na tekstu unutar linka <a> tag, specifično za linkove), **Partial Link Text lokator** (omogućava traženje samo dijela teksta unutar linka i koristi se kada nije moguće navesti cijeli tekst linka), **CSS selector lokator** (koristi CSS selektore za pronalazak elemenata), **XPath lokator** (omogućuje vrlo precizno lociranje elemenata na temelju njihovih atributa, strukture u DOM-u i tekstualnog sadržaja).

Najčešće je korišten ID lokator, jer je najbrži, najprecizniji, jedinstven za svaki element na stranici i najmanje je podložan promjenama unutar strukture stranice.

65. Objasnite glavnu razliku između XPath i ID Selenium lokatora.

Glavna je razlika u tome što je **ID** uvijek jedinstven za element na stranici, a to omogućuje bržu i precizniju selekciju, dok **XPath** ne mora uvijek biti jedinstven te omogućuje fleksibilniju selekciju temeljem različitih uvjeta i strukture stranice, a zbog toga je i sporiji.

66. Ukratko objasnite postupak testiranja web aplikacije.

Za početak, potrebno je analizirati zahtjeve testa i isplanirati testiranje. U toj fazi definiraju se strategije i opsezi testiranja, uključujući i odabir alata i vrsta testiranja. Nakon toga se postavlja okolina i onda se izvode testovi. Testovi posebno gledaju na funkcionalnost aplikacije, izvedbu, upotrebljivost, sigurnost, kompatibilnost s različitim preglednicima i operacijskim sustavima, API-je, mrežne probleme i slično. Nakon dovršetka testova, sastavlja se izvješće o pogreškama. Testiranje web aplikacija može biti statičko i dinamičko. Statičko se odnosi kada je sadržaj na zaslonu isti kao i sadržaj unutar poslužitelja, dok je kod dinamičkog važna komunikacija između poslužitelja kako bi web aplikacije pružile svoje značajke.

67. Ukratko objasnite postupak automatiziranja testiranja web korisničkog sučelja pomoću Selenium WebDriver-a, kako se on koristi i može li se on koristiti za testiranje programske podrške mobilnih uređaja.

Da bismo automatizirali testiranje web korisničkog sučelja pomoću Selenium WebDrivera prvo je potrebno instalirati Selenium i odgovarajući WebDriver za preglednik koji želimo testirati (npr. ChromeDriver za Chrome). Zatim postavljamo okruženje za testiranje, što uključuje instalaciju nekog programskog jezika i biblioteka za Selenium. Nakon toga se kreiraju testne skripte koristeći WebDriver API u kojem se specificiraju koraci (npr. otvaranje URL-a, interakcija s nekim elementima stranice i provjera rezultata). Testovi se pokreću i automatski se bilježe rezultati i eventualne greške. **Selenium WebDriver se koristi za testiranje programske podrške mobilnih uređaja**, ali u kombinaciji s drugim alatima, poput Appiuma. Da bi se koristio, prvo se instalira Selenium WebDriver, nakon toga se instalira Appium koji se integrira sa Selenium WebDriverom, pokrene se appium server, postavi se okruženje za Android/iOS, konfigurira se WebDriver tako da radi s mobilnim uređajima i nakon toga moguće je vršiti testiranje.

68. Navedite osnovna obilježja, način rada i korištenje alata TestNG.

TestNG framework je za testiranje u programskom jeziku Java, osmišljen da podržava širok spektar testiranja uključujući unit, integration, functional i slično. **TestNG** koristi anotacije poput @Test, @BeforeMethod, @AfterMethod i druge za upravljanje testovima. Omogućuje grupiranje testova, prioritizaciju i upravljanje ovisnostima između testova,

generira detaljan HTML i XML izvještaj o izvršenim testovima, podržava paralelno testiranje, jednostavno se integrira s drugim alatima.

Način rada: TestNG koristi Java klase i anotacije za definiranje testnih metoda. On prati unaprijed definiran životni ciklus testova, izvršavajući pripremne metode prije svakog testa i metode za čišćenje nakon svakog testa, testovi se mogu pokretati kroz IDE ili komandnu liniju.

Korištenje: Ako koristimo Maven, unutar pom.xml kao ovisnost moramo dodati TestNG kako bismo ga koristili. Nakon toga, definiramo testove s anotacijom @Test u Java klasi. Testovi se pokreću ili kroz IDE ili kroz komandnu liniju.

69. Navedite neke od najvažnijih TestNG anotacija (annotations) i objasniti njihovo značenje.

@BeforeMethod – metoda ispod biti će izvršena prije svake @Test metode. Koristi se za pripremu okruženja prije svake @Test metode

@AfterMethod – metoda koja će se izvršiti nakon svake @Test metode. Koristi se za čišćenje resursa ili resetiranje stanja.

@BeforeClass – metoda ispod će se izvesti samo jednom prije pokretanja prve metode označene s @Test u toj klasi. Koristi se za inicijalizaciju resursa koji su potrebni za cijelu klasu.

@AfterClass – test ispod bit će izvršen nakon što su izvedene sve metode u toj klasi. Koristi se za oslobađanje resursa ili zatvaranje konekcije nakon završetka svih testova u klasi.

@BeforeTest - bit će izvršeno prije prve @Test metode unutar <test> taga u XML konfiguraciji, može se izvršiti više puta. Koristi se za pripremu okruženja specifičnog za određeni testni skup.

@AfterTest – metoda s ovom anotacijom bit će izvršena nakon svih metode označenih s @Test unutar <test> taga u XML konfiguraciji. Koristi se za čišćenje okruženja specifičnog za određeni testni skup.

70. Ukratko objasnite čemu služi @BeforeMethod i što se definira datotekom testng.xml.

@BeforeMethod koristi se za izvršavanje pripremnih radnji prije svake metode označene s @Test. Omogućava postavljanje početnog stanja za svaki pojedini test te osigurava izolaciju testova tako da promjene nastale u jednom testu ne utječu na druge testove. Konkretno, može se koristiti za resetiranje podataka (brisanje prethodnih unosa iz baze podataka, vraćanje aplikacije u početno stanje i slično), za pripremu okruženja (otvaranje preglednika ili uspostavljanje veze s API-jem).

testng.xml služi kao konfiguracijska datoteka za TestNG koja omogućava organizaciju i upravljanje testovima (moguće je definirati više test slučajeva ili skupova testova u jednoj datoteci), izvršavanje određenih klasa, definiranje redoslijeda izvršavanja testova, postavljanje parametara za testove, podršku za paralelno testiranje.

71. Navedite osnovna obilježja, način rada i korištenje alata Maven.

Maven je alat koji je namijenjen za organizaciju softvera pisanog u Javi upravljajući cijelim životnim ciklusom builda. Temelji se na POM (Project Object Model) koji definira konfiguraciju projekta. Maven koristi unaprijed definiranu hijerarhiju direktorija i standardizirane konvencije, upravlja ovisnostima i podržava različite pluginove za proširivanje funkcionalnosti.

Maven radi kroz životni ciklus builda koji uključuje validaciju da je projekt ispravno konfiguriran, kompajliranje izvornog koda, pokretanje testova, pakiranje projekta,

provjera zadovoljava li projekt zadane uvjete, instalacija paketa u lokalni repozitorij, postavljanje paketa u udaljeni repozitorij, tj. deployment.

Maven integrira testiranje unutar životnog ciklusa builda pomoću alata kao što su JUnit i TestNG. **Da bismo testirali unutar Mavena**, prvo je potrebno dodati testne ovisnosti i pluginove unutar POM datoteke, nakon toga se pokreću testovi automatski u testnoj fazi životnog ciklusa builda, a generiranje izvještaja o testiranju postiže se uz pomoć pluginova kao što je Surefire.

72. Ukratko objasnite čemu služi datoteka POM.xml, te što je i čemu služi Rest Assured.

pom.xml (Project Object Model) glavna je konfiguracijska datoteka unutar Maven projekta. Ona opisuje osnovne informacije o projektu, upravlja ovisnostima o datotekama i pluginovima i definira tijek builda projekta.

Rest Assured je Java biblioteka namijenjena za automatizirano testiranje RESTful API-ja. Koristi se za jednostavno slanje HTTP zahtjeva i provjeru odgovora, čime omogućuje učinkovito testiranje API-ja. Podržava razne HTTP metode i omogućuje jednostavno rukovanje JSON i XML formatima podataka.

73. Objasnite za što se koristi anotacija @Data provider.

@DataProvider je anotacija unutar TestNG frameworka koja se koristi za parametrizaciju testova, omogućujući izvođenje jednog testnog slučaja s različitim skupovima podataka, čime se olakšava testiranje s više ulaznih vrijednosti bez dupliciranja koda.

74. Ukratko objasnite što je *.apk, čemu služi AVD manager, za što se koristi naredba "adb devices", te čemu služi "Desired Capabilities".

***.apk** (Android Package) datotečni je format koji se koristi za instalaciju aplikacija na Android uređaje. Sadrži sve potrebne datoteke za pokretanje aplikacije, kao što su izvorni kodove, resursi, biblioteke i manifest koji definira osnovne informacije o aplikaciji (dozvole i slično). APK datoteke mogu se instalirati na Android uređaje ili na emulator.

AVD Manager (Android Virtual Device Manager) alat je unutar Android Studio-a koji omogućuje kreiranje i upravljanje emulatorima Android uređaja. Emulatori mogu biti u različitim verzijama Androida i različitim konfiguracijama uređaja i slično. Koristi se za testiranje rada aplikacije.

adb devices je naredba koja se koristi za prikaz svih Android uređaja (i fizičkih i virtualnih) koji su povezani s računalom putem ADB-a (Android Debug Bridge). Naredba omogućuje korisnicima da provjere jesu li im uređaji ispravno povezani i spremni za testiranje aplikacija.

Desired Capabilities postavke su koje se koriste u automatizaciji testova za precizno definiranje željenih parametara okruženja u kojem će se testovi izvršavati. To uključuje platformu na kojoj će se testiranje izvoditi, verzija operacijskog sustava, je li riječ o fizičkom uređaju ili emulatoru i slično.

75. Ukratko objasnite postupak testiranja programske podrške mobilnih uređaja.

U testiranju programske podrške mobilnih uređaja obično se testiranju funkcionalnosti, performanse, sigurnost i korisničko iskustvo mobilnih aplikacija na različitim uređajima i operacijskim sustavima. Cijeli proces podijeljen je u faze: **planiranje i priprema testova** u kojoj se definiraju ciljevi testiranja, odaberu se odgovarajući uređaji i operacijski sustavi te se odluči koji će se tipovi testiranja provesti i u kojim alatima i okruženjima. U sljedećoj fazi **aplikacija se instalira** na odabrane uređaje i emulator i nakon toga **se izvode testovi** koji su prethodno definirani. Testove je moguće **automatizirati** pomoću alata kao što je Appium, a time možemo brže detektirati bugove i imati bolju učinkovitost. Na kraju se **analiziraju rezultati i radi se izvještaj**.

76. Navedite osnovna obilježja i način rada Appiuma, objasnite što je Appium sesija, te što Appium koristi za automatiziranje aplikacija.

Appium je open-source framework namijenjen automatiziranom testiranju nativnih, hibridnih i web mobilnih aplikacija koji podržava Android i iOS platforme. Vrlo je popularan zbog svoje fleksibilnosti, jer omogućuje pisanje testova u više programskih jezika, automatiziranje testiranja na više uređaja istovremeno te ne zahtijeva instalaciju dodatnih SDK-ova na uređajima. **Način rada** temelji se na korištenju WebDrivera koji omogućuje interakciju s mobilnim uređajem ili emulatorom. Appium pokreće testove koristeći različite platforme koji omogućuju kontrolu aplikacija kroz API-je.

Appium sesija predstavlja interakciju između testnog skriptnog koda i aplikacije na mobilnom uređaju. To je osnovna jedinica koja omogućuje Appiumu pokretanje i kontrolu aplikacije, da komunicira s aplikacijom i uređajem i omogućuje izvještavanje o rezultatima testova. Započinje pokretanjem testa i završava završetkom testa, pa je svaka sesija jedinstvena.

Za automatiziranje aplikacija, Appium se oslanja na **WebDriver** protokol, koji je standardizirani API za upravljanje aplikacijama i preglednicima. On omogućava slanje naredbi uređaju kao što su klikanje na elemente, unos teksta i slično. Koristi **UIAutomator** za Android i **XCUITest** za iOS. Također, koristi se **Appium Server** koji je centralni dio alata i omogućuje svim testovima, pa tako i automatiziranim, da komunikaciju između testova i uređaja putem REST API-ja.

77. Navedite osnovnu razliku između Selenium WebDrivera i Appiuma.

Osnovna je razlika da je **Selenium WebDriver** alat dizajniran za automatizaciju testiranja web aplikacija na desktop preglednicima, kao što su Chrome, Firefox, Edge i drugi, dok **Appium** omogućuje automatizaciju testiranja isključivo mobilnih aplikacija, neovisno o tome jesu li nativne, hibridne ili web aplikacije.

78. Navedite osnovna obilježja, način rada i namjenu (vrste testiranja) JMetera, te koje programske jezike koristi/podržava.

JMeter je open-source alat za testiranje performansi aplikacija i to uglavnom web aplikacija. **JMeter** podržava testiranje aplikacija koje koriste različite protokole, omogućava prilagođavanje testnih planova putem skriptiranja i pluginova, pruža intuitivno grafičko sučelje za kreiranje i konfiguraciju testova, može se koristiti za jednostavne ili distribuirane testove. **JMeter radi tako da** korisnici prvo kreiraju testni plan koji se sastoji od elemenata kao što su Thread Groups, Samplers, Listeners, Config Elements i Assertions. JMeter pokreće virtualne korisnike koji simuliraju stvarne korisničke aktivnosti prema definiranim parametrima. Na kraju se generiraju rezultati u različitim formatima i tako omogućuje analizu izvedbe sustava.

JMeter namijenjen je testiranju performansi u prvom planu, opterećenja, stres testiranje, a manje je učinkovit u testiranju funkcionalnosti i regresijskom testiranju.

JMeter podržava prvenstveno Javu, ali podržava i neke skriptne jezike kao što su JavaScript, Groovy i BeanShell.

79. Ukratko objasnite čemu služi Thread Group, te što predstavljaju "Number of Threads", "Ramp-Up period", "Loop-Count" u JMeteru?

Thread Group je osnovni element testnog plana koji definira ponašanje virtualnih korisnika koji simuliraju stvarne korisnike tijekom testiranja performansi u JMeteru. Kombinacijom njegovih parametara moguće je fleksibilno modelirati različite scenarije opterećenja za aplikaciju.

Number of Threads parametar je od Thread Group koji predstavlja broj virtualnih korisnika koji će istovremeno izvršavati test. Svaki thread predstavlja neovisnog korisnika koji izvršava definirani testni plan.

Ramp-Up period također je parametar od Thread Group koji predstavlja vrijeme u sekundama tijekom kojeg se svi threadovi postupno aktiviraju.

Loop Count je isto tako parametar od Thread Group koji određuje koliko puta svaki thread treba ponoviti definirani testni plan.

Primjer: Ako postavimo Number Of Threads = 50, Ramp-Up Period = 100 sekundi i Loop Count = 2, tada će rezultat biti 50 virtualnih korisnika koji se postepeno aktiviraju tijekom 100 sekundi (1 korisnik svake 2 sekunde), a svaki korisnik izvršava testni plan 2 puta.

80. Ukratko objasnite čemu služi konfiguracijski element "HTTP request Deafult", te Listener "Graph Results", Listener "View Results Tree" i logički kontroler "Once Only controller".

HTTP Request Default je konfiguracijski element unutar JMetera koji omogućuje postavljanje zadanih vrijednosti za HTTP zahtjeve unutar testnog plana, a za cilj ima smanjiti ponavljanje unosa istih parametara kao što su URL, host, port i slično.

Listener Graph Results služi za vizualizaciju rezultata testa u obliku grafova koji prikazuju informacije o performansama kao što su prosječno vrijeme odgovora, broj zahtjeva u sekundi i slično. Koristi se za praćenje rezultata u stvarnom vremenu i identificiranje trendova tijekom testiranja.

Listener View Results Tree služi za prikazivanje detaljnog pregleda svakog zahtjeva i odgovora uključujući prikaz sadržaja odgovora (HTML, JSON, XML), zaglavlja, parametara i statusa odgovora. Idealan je za debugging, jer omogućuje detaljnu analizu pojedinačnih zahtjeva i odgovora.

Once Only Controller osigurava da se skup akcija ili zahtjeva unutar njega izvrši samo jednom za svakog virtualnog korisnika. Koristi se za zadatke koji trebaju biti izvršeni samo jednom, kao što su login, prije nego se pokrene glavni test

81. Ukratko navedite osnovna obilježja i za što se koriste neki od ostalih alata ili okolina za automatizirano testiranje: Saucelabs, Browsera, Browserstack, Katalon Studio, Espresso.

Saucelabs je mrežna platforma za automatizirano testiranje koje omogućuje testiranje na različitim uređajima i web preglednicima. Podržava testiranje uživo i omogućuje paralelno pokretanje testova s različitim kombinacijama uređaja i preglednika. Nudi pristup privatnim oblacima uređaja za testiranje na stvarnim mobilnim uređajima, s potpunim pristupom i izolacijom testova.

Saucelabs se koristi za ubrzanje testiranja, podizanje kapaciteta testova te simulaciju korisničkih scenarija na širokom spektru uređaja i preglednika.

Browsera je alat za automatizirano testiranje web aplikacija u različitim kombinacijama operacijskih sustava i preglednika. Oponaša akcije testera i uspoređuje snimke zaslona kako bi detektirao razlike između različitih preglednika. Prikazuje detalje o greškama u JavaScript kodu.

Browsera se koristi za vizualno testiranje web stranica na različitim preglednicima, detektiranje grešaka u dizajnu i JavaScript kodu.

BrowserStack usluga je u oblaku za testiranje funkcionalnosti i dizajna web aplikacija na različitim operacijskim sustavima i preglednicima. Omogućuje testiranje uživo i automatizirano testiranje na širokom spektru raspona uređaja, uključujući i računala i mobilne uređaje. Koristi virtualne strojeve za svaku kombinaciju OS-a i preglednika te nudi alate kao što su Selenium i JavaScript za automatizaciju testova.

BrowserStack se koristi za testiranje web aplikacija u različitim okruženjima, omogućujući testiranje uživo i pisanje automatiziranih testova.

Katalon Studio je alat za automatizirano testiranje web i mobilnih aplikacija, podržava snimanje i reprodukciju aktivnosti. Integrira se s alatima kao što su Git, Jenkins i Jira. Podržava Javu i Groovy te nudi predloške za rukovanje repozitorijem objekata i testnim slučajevima.

Katalon Studio koristi se za automatizirano testiranje aplikacija, posebno u web i mobilnim okruženjima.

Espresso je framework za automatizirano testiranje mobilnih aplikacija razvijen od Google-a. Fokusira se na testiranje grafičkog sučelja aplikacija te nudi snažne sinkronizacijske sposobnosti za precizne i brze testove korisničkog sučelja.

Espresso se koristi za automatizirano testiranje korisničkog sučelja mobilnih aplikacija omogućujući programerima brzo testiranje mobilnih sučelja u Android aplikacijama.

82. Objasnite uobičajene korake napada na sigurnost računalnog sustava, najčešće oblike ranjivosti programskog koda, te navedite neke od alata za testiranje sigurnosti programske podrške.

Uobičajeni koraci napada na sigurnost računalnog sustava uključuju pokušaj napada na sustav, gdje napadač započinje istraživanje sustava kako bi identificirao potencijalne slabosti i načine za napad. Napadač otkrije ranjivost sustava, kao što su sigurnosne pogreške u aplikacijama, loša konfiguracija i slično. Kada napadač uspješno iskoristi ranjivost, može proći kroz sigurnosne kontrole sustava i pristupiti osjetljivim podacima sustava. Tada napadač preuzima kontrolu nad sustavom, što mu omogućuje izvršavanje zlonamjerne radnje. U toj fazi, napadač može iskoristiti sustav za prikupljanje podataka, ubacivanje malicioznog softvera ili neovlašteno upravljanje.

Najčešći oblici ranjivosti su ubrizgavanje (tehnika u kojoj napadači šalju zlonamjerne podatke kako bi izvršili neautorizirane naredbe u aplikaciji. Vjerojatno najpoznatije je SQL ubrizgavanje), neispravna autentikacija (kada aplikacija neispravno provodi autentikaciju, napadači mogu preuzeti korisničke račune i pristupiti sustavu), izlaganje osjetljivih podataka (uglavnom se događa zbog toga jer sustavi nisu enkriptirani), XML parseri, loša kontrola pristupa, pogrešna konfiguracija sigurnosti, cross-site scripting (izvršavanje zlonamjerne skripte u web pregledniku)...

Popularni alati za testiranje sigurnosti programske podrške su Arachni, Vega, OWASP ZAP, w3af i Nikto.

83. Objasnite što je, za koje primjere primjene i s kojim ciljem provodi testiranje otpornosti na stres, te usporedite ukratko testiranje opterećenja i testiranje otpornosti na stres.

Testiranje otpornosti na stres vrsta je testiranja programa kojim se provjerava stabilnost i pouzdanost aplikacije. **Cilj takvog testiranja** je mjerenje robusnosti programa i sposobnosti rukovanja pogreškama u uvjetima iznimno velikog opterećenja i osiguravanje da se program ne sruši u kritičnim situacijama. **Najistaknutija upotreba testiranja otpornosti na stres** je određivanje granice na kojoj se sustav ili program ili sklopovski dio kvare. Konkretni primjer kada se takvo testiranje provodi je za vrijeme nekog događaja, prodaje karata, rasprodaje, kada se blog spomene u vodećim novinama pa doživi nagli porast prometa.

Testiranje opterećenja služi za testiranje ponašanja sustava u normalnim uvjetima radnog opterećenja i samo je testiranje ili simulacija s stvarnim radnim opterećenjima, rad sustava se ne prekida. **Testiranje otpornosti na stres** testira ponašanje sustava u

ekstremnim uvjetima i provodi se do kvara sustava. Pokušava se srušiti sustav testiranjem s ogromnom količinom podataka ili zahtjeva za resursima.

84. Navedite i kratko objasnite glavne tipove testiranja na stres, korake provedbe i najznačajnije alate.

Raspodijeljeno testiranje otpornosti na stres – u sustavima klijent-server, stres testiranje provodi se na svim klijentima uz pomoć servera koji raspodjeljuje testove i prati status svakog klijenta. Server počinje slati podatke za testiranje nakon što klijent uspostavi vezu, a klijentski uređaji šalju signale kako bi označili da su povezani. Ako server ne prima signal od klijenta, potrebno je dodatno ispitivanje sustava.

Testiranje aplikacije na stres – fokusira se na pronalazak problema povezanih sa zaključavanjem podataka, blokiranjem podataka, mrežnim problemima i uskim grlima u izvedbi unutar sustava.

Testiranje transakcija na otpornost na stres – provodi se testiranje transakcija između više aplikacija kako bi se pronašle slabosti u obradi transakcija i optimizaciji sustava.

Sustavsko testiranje otpornosti na stres – testira otpornost više sustava koji rade na istom serveru, otkrivajući probleme gdje jedan sustav može blokirati drugi, npr. zbog sukoba u pristupu podacima.

Eksploratorno testiranje otpornosti na stres – koristi neobične uvjete i parametre koji se rijetko javljaju u stvarnim scenarijima, npr. ekstremno velik broj korisnika prijavljen u isto vrijeme, istovremeni unos velike količine podataka u bazu itd. Cilj je identificirati slabosti u neočekivanim situacijama.

Koraci provedbe testiranja otpornosti na stres su **planiranje testa otpornosti na stres** (prikupljanje podataka o sustavu i definiranje ciljeva testiranja otpornosti na stres), **izrada skripti za automatizaciju** (stvaranje skripti za automatizaciju testiranja otpornosti na stres, generiranje testnih podataka za scenarije stresa), **izvršenje skripte** (pokretanje skripte za automatizaciju testiranja stresa i pohrana rezultata testa), **analiza rezultata** (analiza i identifikacija uskih grla), **ugađanje i optimizacija** (fino podešavanje sustava, mijenjanje konfiguracije, optimiranje koda s ciljem ispunjavanja željenih mjerila).

Alati koji se često koriste su: **LoadRunner** (HP-ov alat široko korišten za testiranje opterećenja, rezultati su mu referentni u industriji), **JMeter**, **Stress Tester** (nudi opsežnu analizu performansi web aplikacija, uključujući jednostavno pisanje skripti i prikazivanje rezultata u grafičkom formatu), **NeoLoad** (popularan alat za testiranje performansi web i mobilnih aplikacija. Može simulirati tisuće korisnika i analizirati odgovore sustava pod opterećenjem, podržava testiranje u oblaku, što ga čini skalabilnim i isplativim).

85. Prokomentirajte glavne trendove u testiranju programske podrške.

QAOps (Quality Assurance Operations) – pristup koji omogućuje integraciju QA u svaki korak razvoja i isporuke programske podrške. Omogućuje bržu isporuku i identifikaciju problema u ranoj fazi, pa se smanjuje vrijeme potrebno za ispravljanje grešaka.

Automatizirano testiranje bez skripti – omogućuje brže testiranje i smanjuje potrebu za tehničkim znanjem. Koriste se vizualni alati i sučelja.

Testiranje mikrousloga – ključno je u modernim arhitekturama, jer se aplikacije sastoje od niza malih i neovisnih servisa koji komuniciraju među sobom, a testira se svaki servis zasebno, kao i njihova integracija.

Testiranje AI i ML zasnovanih rješenja – testiranje rješenja temeljenih na umjetnoj inteligenciji i strojnom učenju predstavlja izazov zbog njihove nepredvidljivosti i kompleksnosti. Testiranje uključuje validaciju podataka, provjeru prediktivnih modela i preciznost modela.